# Hashing

Sebastian Mestre

Octubre 2026

# Definición

Un hash es una representación de tamaño fijo de otro objeto.

Sirve para:

- comparar dos objetos en $O(1)$,
- usarlo como clave de un arreglo o de un diccionario,
- hacer sobre el hash una cuenta que imita una operación del objeto.

# Hash polinomial

Cada caracter es un entero entre 1 y $|\Sigma|$. Si el 0 fuera un caracter, las strings 0, 00, 000 tendrían todas hash 0. Elegimos un entero $b > |\Sigma|$ y un primo $p$ grande.

$$h(s) = \sum_{i=0}^{n-1} s[i]\, b^i \pmod{p}, \qquad n = |s|.$$

Si $h(s_1) = h(s_2)$, tratamos a $s_1$ y $s_2$ como iguales.

Cuando $s_1 = s_2$, los hashes coinciden. Cuando $s_1 \neq s_2$, coinciden muy pocas veces.

# Substrings en $O(1)$

$[l, r)$ no incluye el extremo derecho

Muchos problemas piden comparar substrings de una misma string grande.

$$
\begin{aligned}
h(s[l, r)) &= s[l]\, b^0 + s[l+1]\, b^1 + \cdots + s[r-1]\, b^{r-l-1} \\
h(s[l, n)) &= s[l]\, b^0 + s[l+1]\, b^1 + \cdots + s[r-1]\, b^{r-l-1} + s[r]\, b^{r-l} + s[r+1]\, b^{r-l+1} + \cdots \\
h(s[r, n))\, b^{r-l} &= \qquad\qquad + s[r]\, b^{r-l} + s[r+1]\, b^{r-l+1} + \cdots
\end{aligned}
$$

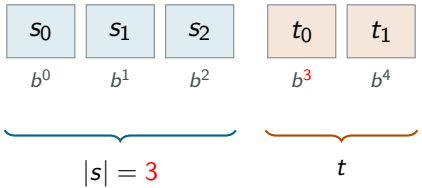
$$
h(s[l, r)) = h(s[l, n)) - h(s[r, n))\, b^{r-l}
$$

Con $h(s[i, n))$ y $b^i$ ya calculados, para todo $i$, cada substring sale en $O(1)$.

# Concatenar

De $(h(s), |s|)$ y $(h(t), |t|)$ sale el hash de $st$, sin construir la string.

$$h(st) = h(s) + h(t)\, b^{|s|} \qquad |st| = |s| + |t|$$



# El par $(h, \text{largo})$

Para igualdad y concatenación, el par hace de string.

- Igualdad: se comparan los dos números, en $O(1)$.
- Concatenación: la cuenta de arriba, en $O(1)$.

Esta representación de la string no permite leer un caracter.

La idea es más general: se puede construir una representación que reemplace al objeto, y operar sobre esa representación.

TAP 2019, problema H, Rummaging.

`https://codeforces.com/group/YjFmW2O15Q/contest/102363/problem/H`

# Cuántas colisiones

Modelamos cada hash como un número uniforme entre 0 y $2^{64} - 1$. Dos elementos distintos colisionan con probabilidad $2^{-64}$.

Sea $X$ la cantidad de colisiones entre $N$ elementos. Por linealidad de la esperanza:

$$\mathbb{E}[X] = \binom{N}{2} 2^{-64}.$$

$X$ es un entero no negativo, así que $\mathrm{P}(X \neq 0) \leq \mathbb{E}[X]$.

Para $N = 10^6$,

$$\mathrm{P}(X \neq 0) \leq \binom{10^6}{2} 2^{-64} \approx 2.71 \times 10^{-8}.$$

# Bases y módulos

Para una base y un módulo fijos se pueden armar strings con muchas colisiones. Ese caso de prueba tiene que estar construido para esa base.

En Latam eso casi no aparece. Una base grande, elegida al azar, corta los tests que necesitan conocer $b$.

Thue-Morse colisiona módulo $M$ cuando $b^{2^k} \equiv 1 \pmod{M}$ para algún $k$. El módulo no tiene que ser una potencia de 2. Si es $2^{64}$ y la base es impar, colisiona igual. La vemos al final.

# Hashing de conjuntos

# XOR hashing

Conjuntos de $\{0, \ldots, n-1\}$. Los $r[i]$ son números aleatorios, elegidos una vez.

$$
\begin{aligned}
h(\emptyset) &= 0 \\
h(\{i\}) &= r[i] \\
h(A \triangle B) &= h(A) \oplus h(B)
\end{aligned}
$$

$$
\begin{aligned}
h(\{a, b, c\}) &= h(\{a\} \triangle \{b\} \triangle \{c\}) \\
&= r[a] \oplus r[b] \oplus r[c]
\end{aligned}
$$

Asociativa, conmutativa, con neutro 0. Cada conjunto es su inverso: $h(A) \oplus h(A) = 0$.

# Insertar y borrar

Como $r[i] \oplus r[i] = 0$, agregar y sacar son la misma cuenta.

Agregar $i$, sabiendo que no está:

$$h \leftarrow h \oplus r[i]$$

Sacar $i$, sabiendo que está:

$$h \leftarrow h \oplus r[i]$$

Si $i$ ya estaba y se agrega, el XOR lo saca. El hash no dice si $i$ pertenece al conjunto.

# Sum hashing

La misma idea, ahora con multiconjuntos. Los $r[i]$ son otra vez una tabla aleatoria.

$$h(\emptyset) = 0$$
$$h(\{i\}) = r[i]$$
$$h(A + B) = h(A) + h(B)$$
$$h(A \setminus B) = h(A) - h(B)$$

La resta corresponde a la diferencia cuando $B$ está contenido en $A$, contando repeticiones.

$h(\{a, a, b\}) = 2\,r[a] + r[b]$.

Práctica: TC ARG 2024, día 7, problema F.

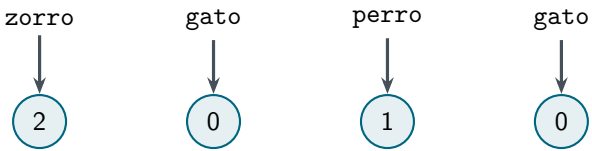
`https://codeforces.com/gym/104686/problem/F`

# Hashing con orden

# Comprimir

Hay que hacer $N^2$ comparaciones entre $N$ strings, con $N = 2000$ y cada string de largo 2000.

Se ordenan una vez. Cada string pasa a ser su índice en ese orden, y comparar es comparar enteros.




Orden de los distintos: `gato`, `perro`, `zorro`. Eso es comprimir: ordenar, borrar repetidos, reemplazar por el índice.

# Concatenar ids

$A$ son strings de largo 3, $B$ de largo 2. Los ids conservan el orden adentro de cada conjunto.

| | | |
|---|---|---|
| $A$ | `gat` | `perr` |
| | 0 | 1 |
| $B$ | `as` | `os` |
| | 0 | 1 |

| par | concatenación | id |
|---|---|---|
| $(0,0)$ | `gatas` | 0 |
| $(0,1)$ | `gatos` | 1 |
| $(1,0)$ | `perras` | 2 |
| $(1,1)$ | `perros` | 3 |

El orden de los pares es el orden de las concatenaciones. Esos pares se comprimen, y los ids nuevos quedan ordenados como las strings $ab$.

# Concatenar ids

El corte entre las dos partes tiene que caer siempre en el mismo lugar. Si los largos de $A$ no son todos iguales, o los de $B$, el par deja de representar a la concatenación.

$$\texttt{a} \to 0,\ \texttt{ab} \to 1 \quad \texttt{a+bc} \quad \texttt{abc} \quad \text{par } (0,0)$$
$$\texttt{bc} \to 0,\ \texttt{c} \to 1 \quad \texttt{ab+c} \quad \texttt{abc} \quad \text{par } (1,1)$$

Las dos concatenaciones son la misma string. Los pares no, y al comprimirles salen ids distintos.

# Suffix array

Dada una string de largo $n$, imprimir los índices donde empiezan sus sufijos, en orden lexicográfico.

| índice | sufijo |
|---|---|
| 0 | caba |
| 1 | aba |
| 2 | ba |
| 3 | a |

Orden:

a, aba, ba, caba

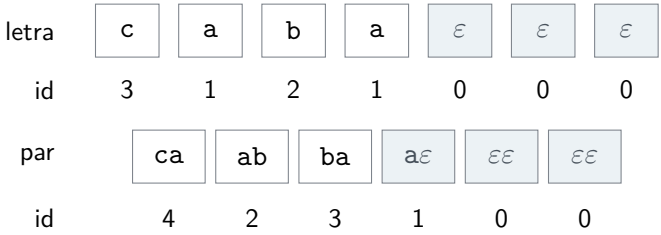
Resultado: 3 1 2 0

https://judge.yosupo.jp/problem/suffixarray

# Suffix array

Se arman ids para los substrings de largo 1. Esos se concatenan y se comprimen: salen los ids de largo 2, después los de largo 4, y así.

Antes se agrega padding al final, para que los pares no se salgan de la string.



# Suffix array

$N$ es el largo de la string con el padding.

```
ids ← comprimir los caracteres
l ← 1
mientras l < n:
  par ← [(ids[i], ids[i + l]) for i = 0 ..< N − 2l + 1]
  ids ← comprimir par
  l ← 2l
```

Al terminar, el sufijo que empieza en $i$ queda en la posición $\mathrm{ids}[i]$. El arreglo que hay que imprimir es la inversa: $\mathrm{SA}[\mathrm{ids}[i]] = i$.

En caba, después de la vuelta $l = 2$, los ids son $3, 1, 2, 0$. La inversa es 3 1 2 0.

# Problemas

- **Igualdad de caminos en un árbol.** Un caracter en cada arista, y consultas $(u_1, v_1, u_2, v_2)$. La string de un camino es la concatenación de las aristas, de $u$ a $v$. Decir si los dos caminos forman la misma string.
- **Comparación de caminos en un árbol.** Las mismas consultas. Si las strings son distintas, decir cuál es menor.
- **Árboles isomorfos.** Dos árboles. Decidir si son isomorfos. Se puede resolver sin hashing. Con sum hashing, la implementación queda más corta.
- **ABC 481 F.** Grafo dirigido, un dígito del 1 al 9 en cada arista. El costo de un camino es el número que forman esos dígitos. Para cada nodo, el menor costo desde el nodo 1, módulo 998244353.

  `https://atcoder.jp/contests/abc481/tasks/abc481_f`

# Problemas

- **TAP 2019 H, Rummaging.** Un texto, y varias frases del mismo largo. Contar cuántas veces aparecen en el texto, permitiendo a lo sumo un caracter distinto.
  `https://codeforces.com/group/YjFmW2O15Q/contest/102363/problem/H`
- **TC ARG 2024, día 7, F.** $N$ strings del mismo largo, con caracteres A, B, C y D. Hay una sola cuya cantidad de posiciones distintas con cada una de las demás es $K$. Encontrarla.
  `https://codeforces.com/gym/104686/problem/F`

Gracias

# Apéndice: Thue-Morse

# Thue-Morse

$\tau_0 = 0$. Cada término es el anterior, seguido de su complemento.

$\tau_1$ 0 1

$\tau_2$ 01 10

$\tau_3$ 0110 1001

$\tau_4$ 01101001 10010110

la primera mitad, que es $\tau_{i-1}$ su complemento

# Por qué colisionan

$W$ es el hash polinomial sin tomar módulo. La segunda mitad de $\tau_n$ es el complemento de la primera, así que

$$W(\overline{\tau_n}) - W(\tau_n) = \prod_{i=0}^{n-1} (1 - b^{2^i}).$$

Si $b^{2^k} \equiv 1 \pmod{M}$ para algún $k < n$, ese factor es 0 módulo $M$. Los dos hashes coinciden. El módulo puede ser cualquiera.

Si $M = 2^{64}$ y $b$ es impar, cada factor aporta potencias de 2: $2^{i+1}$ divide a $1 - b^{2^i}$, y el producto junta al menos $n(n+1)/2$ factores 2. Con $n = 11$ eso pasa de 64. $\tau_{11}$ y su complemento, de largo 2048, colisionan para cualquier base impar.

# Un ejemplo

Base $b = 3$, módulo $2^{10} = 1024$, con $n = 4$.

$$0110\ \ 1001\ \ 1001\ \ 0110$$
$$1001\ \ 0110\ \ 0110\ \ 1001$$

Los dos hashes valen 464.

Otro módulo, que no es potencia de 2: base $b = 4$, módulo 15. Como $4^2 \equiv 1 \pmod{15}$, $\tau_2 = 0110$ y su complemento 1001 tienen hash 5.

# Cómo generarlas

```cpp
vector<int> thue(int n) {
    vector<int> s = {0};
    for (int k = 0; k < n; k++) {
        int m = (int) s.size();
        for (int i = 0; i < m; i++)
            s.push_back(1 - s[i]);
    }
    return s;
}

uint64_t H(vector<int> const& s, uint64_t b, uint64_t mod) {
    uint64_t h = 0, p = 1;
    for (int x : s) {
        h = (h + (uint64_t) x * p) % mod;
        p = p * b % mod;
    }
    return h;
}
// H(thue(4), 3, 1024) y el hash del complemento valen 464
```

# Apéndice: solución al ABC 481 F

# El orden de los costos

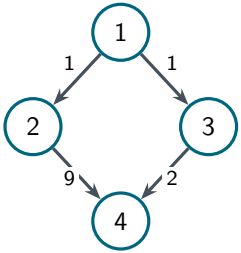
Un camino con más aristas tiene más dígitos. Con dígitos del 1 al 9, ese número es mayor. Repetir un nodo agrega dígitos, así que no mejora el mínimo.

Entre dos caminos con la misma cantidad de aristas, el menor número es el menor en orden lexicográfico.

Alcanza con buscar el camino lexicográficamente menor entre los de longitud mínima.

# Capa por capa

El nodo 1 empieza con la string vacía, id 0. En la capa $k$ se extienden los caminos de la capa $k-1$ con una arista, y cada nodo se queda con el menor par $(\mathrm{id}, d)$. Esos pares ganadores se comprimen, y los ids nuevos se usan en la capa siguiente.




Capa 1. Desde 1 llegan a 2 y a 3 el par $(0, 1)$.
Costo 1. Los dos quedan con id 0.

Capa 2. Hacia 4:

| desde | par | costo |
|---|---|---|
| 2, dígito 9 | $(0, 9)$ | 19 |
| 3, dígito 2 | $(0, 2)$ | 12 |

Gana $(0, 2)$. El nodo 4 se queda con 12.